

Create S3 Bucket:

s3-openweather-oh

Open Weather API Key:

<https://openweathermap.org/>

- Create your account

https://home.openweathermap.org/api_keys

- Copy your Active Key

IAM Role

airflow-openweather-role

- AmazonEC2FullAccess
- AmazonS3FullAccess

IAM USER

airflow-openweather-user

- AdministratorAccess

Create access key

- Check → Command Line Interface (CLI) – & – Confirmation
- Copy your [Access Key ID](#) & [Secret Access Key](#)

EC2

Create EC2 Instance

- t2.xlarge
- Port (4000-38888) → Anywhere (0.0.0.0/0)
- Configure Storage (30 GB)
- airflow-openweather-role

Create a Folder

```
├── docker_exp
│   ├── Dockerfile
│   └── docker-compose.yaml
```

Dockerfile

```
FROM apache/airflow:3.1.0
```

```
# Install libraries needed for S3 and Pandas S3 support
```

```
RUN pip install --no-cache-dir s3fs boto3
```

docker-compose.yml

```
# Licensed to the Apache Software Foundation (ASF) under one
# or more contributor license agreements. See the NOTICE file
# distributed with this work for additional information
# regarding copyright ownership. The ASF licenses this file
# to you under the Apache License, Version 2.0 (the
# "License"); you may not use this file except in compliance
# with the License. You may obtain a copy of the License at
#
# http://www.apache.org/licenses/LICENSE-2.0
#
# Unless required by applicable law or agreed to in writing,
# software distributed under the License is distributed on an
# "AS IS" BASIS, WITHOUT WARRANTIES OR CONDITIONS OF ANY
# KIND, either express or implied. See the License for the
# specific language governing permissions and limitations
# under the License.
#
# Basic Airflow cluster configuration for CeleryExecutor with Redis and PostgreSQL.
#
# WARNING: This configuration is for local development. Do not use it in a production deployment.
#
# This configuration supports basic configuration using environment variables or an .env file
# The following variables are supported:
#
# AIRFLOW_IMAGE_NAME      - Docker image name used to run Airflow.
#                          Default: apache/airflow:3.1.0
# AIRFLOW_UID             - User ID in Airflow containers
#                          Default: 50000
# AIRFLOW_PROJ_DIR        - Base path to which all the files will be volumed.
#                          Default: .
# Those configurations are useful mostly in case of standalone testing/running Airflow in test/try-out mode
#
# _AIRFLOW_WWW_USER_USERNAME - Username for the administrator account (if requested).
#                             Default: airflow
# _AIRFLOW_WWW_USER_PASSWORD - Password for the administrator account (if requested).
#                             Default: airflow
# _PIP_ADDITIONAL_REQUIREMENTS - Additional PIP requirements to add when starting all containers.
#                               Use this option ONLY for quick checks. Installing requirements at container
#                               startup is done EVERY TIME the service is started.
#                               A better way is to build a custom image or extend the official image
#                               as described in https://airflow.apache.org/docs/docker-stack/build.html.
#                               Default: "
#
# Feel free to modify this file to suit your needs.
---
x-airflow-common:
  &airflow-common
  # In order to add custom dependencies or upgrade provider distributions you can use your extended image.
  # Comment the image line, place your Dockerfile in the directory where you placed the docker-compose.yml
  # and uncomment the "build" line below, Then run `docker-compose build` to build the images.
  image: ${AIRFLOW_IMAGE_NAME:-apache/airflow:3.1.0}
  build: .
  environment:
    &airflow-common-env
    AIRFLOW__CORE__EXECUTOR: CeleryExecutor
    AIRFLOW__CORE__AUTH_MANAGER: airflow.providers.fab.auth_manager.fab_auth_manager.FabAuthManager
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN: postgresql+psycopg2://airflow:airflow@postgres/airflow
    AIRFLOW__CELERY__RESULT_BACKEND: db+postgresql://airflow:airflow@postgres/airflow
    AIRFLOW__CELERY__BROKER_URL: redis://:@redis:6379/0
    AIRFLOW__CORE__FERNET_KEY: "
    AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
    AIRFLOW__CORE__LOAD_EXAMPLES: 'true'
    AIRFLOW__CORE__EXECUTION_API_SERVER_URL: 'http://airflow-apiserver:8080/execution/'
  # yamllint disable rule:line-length
  # Use simple http server on scheduler for health checks
  # See https://airflow.apache.org/docs/apache-airflow/stable/administration-and-deployment/logging-monitoring/check-health.html#scheduler-health-check-server
```

```

# yamllint enable rule:line-length
AIRFLOW__SCHEDULER__ENABLE_HEALTH_CHECK: 'true'
# WARNING: Use _PIP_ADDITIONAL_REQUIREMENTS option ONLY for a quick checks
# for other purpose (development, test and especially production usage) build/extend Airflow image.
_PIP_ADDITIONAL_REQUIREMENTS: ${_PIP_ADDITIONAL_REQUIREMENTS:-}
# The following line can be used to set a custom config file, stored in the local config folder
AIRFLOW_CONFIG: '/opt/airflow/config/airflow.cfg'
volumes:
- ${AIRFLOW_PROJ_DIR:-.}/dags:/opt/airflow/dags
- ${AIRFLOW_PROJ_DIR:-.}/logs:/opt/airflow/logs
- ${AIRFLOW_PROJ_DIR:-.}/config:/opt/airflow/config
- ${AIRFLOW_PROJ_DIR:-.}/plugins:/opt/airflow/plugins
user: "${AIRFLOW_UID:-50000}:0"
depends_on:
  &airflow-common-depends-on
  redis:
    condition: service_healthy
  postgres:
    condition: service_healthy
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres-db-volume:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "airflow"]
      interval: 10s
      retries: 5
      start_period: 5s
    restart: always
  redis:
    # Redis is limited to 7.2-bookworm due to licencing change
    # https://redis.io/blog/redis-adopts-dual-source-available-licensing/
    image: redis:7.2-bookworm
    expose:
      - 6379
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 30s
      retries: 50
      start_period: 30s
    restart: always
  airflow-apserver:
    <<: *airflow-common
    command: api-server
    ports:
      - "8080:8080"
    healthcheck:
      test: ["CMD", "curl", "--fail", "http://localhost:8080/api/v2/version"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 30s
    restart: always
    depends_on:
      <<: *airflow-common-depends-on
    airflow-init:
      condition: service_completed_successfully

  airflow-scheduler:
    <<: *airflow-common
    command: scheduler
    healthcheck:
      test: ["CMD", "curl", "--fail", "http://localhost:8974/health"]
      interval: 30s

```

```

    timeout: 10s
    retries: 5
    start_period: 30s
    restart: always
    depends_on:
      <<: *airflow-common-depends-on
    airflow-init:
      condition: service_completed_successfully

airflow-dag-processor:
  <<: *airflow-common
  command: dag-processor
  healthcheck:
    test: ["CMD-SHELL", 'airflow jobs check --job-type DagProcessorJob --hostname "${HOSTNAME}"]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 30s
    restart: always
    depends_on:
      <<: *airflow-common-depends-on
    airflow-init:
      condition: service_completed_successfully

airflow-worker:
  <<: *airflow-common
  command: celery worker
  healthcheck:
    # yamllint disable rule:line-length
    test:
      - "CMD-SHELL"
      - 'celery --app airflow.providers.celery.executors.celery_executor.app inspect ping -d "celery@${HOSTNAME}" || celery --app
airflow.executors.celery_executor.app inspect ping -d "celery@${HOSTNAME}"'
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 30s
  environment:
    <<: *airflow-common-env
    # Required to handle warm shutdown of the celery workers properly
    # See https://airflow.apache.org/docs/docker-stack/entrypoint.html#signal-propagation
    DUMB_INIT_SETSID: "0"
  restart: always
  depends_on:
    <<: *airflow-common-depends-on
  airflow-apserver:
    condition: service_healthy
  airflow-init:
    condition: service_completed_successfully

airflow-triggerer:
  <<: *airflow-common
  command: triggerer
  healthcheck:
    test: ["CMD-SHELL", 'airflow jobs check --job-type TriggererJob --hostname "${HOSTNAME}"]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 30s
    restart: always
  depends_on:
    <<: *airflow-common-depends-on
  airflow-init:
    condition: service_completed_successfully

airflow-init:
  <<: *airflow-common
  entrypoint: /bin/bash
  # yamllint disable rule:line-length
  command:

```

```

-c
- |
if [[ -z "${AIRFLOW_UID}" ]]; then
    echo
    echo -e "\033[1;33mWARNING!!!: AIRFLOW_UID not set!\e[0m"
    echo "If you are on Linux, you SHOULD follow the instructions below to set "
    echo "AIRFLOW_UID environment variable, otherwise files will be owned by root."
    echo "For other operating systems you can get rid of the warning with manually created .env file:"
    echo "   See: https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/index.html#setting-the-right-airflow-user"
    echo
    export AIRFLOW_UID=${$(id -u)}
fi
one_meg=1048576
mem_available=$((($(getconf _PHYS_PAGES) * $(getconf PAGE_SIZE) / one_meg))
cpus_available=$(grep -cE 'cpu[0-9]+' /proc/stat)
disk_available=$((df / | tail -1 | awk '{print $4}'))
warning_resources="false"
if (( mem_available < 4000 )) ; then
    echo
    echo -e "\033[1;33mWARNING!!!: Not enough memory available for Docker.\e[0m"
    echo "At least 4GB of memory required. You have $(numfmt --to iec $(mem_available * one_meg))"
    echo
    warning_resources="true"
fi
if (( cpus_available < 2 )); then
    echo
    echo -e "\033[1;33mWARNING!!!: Not enough CPUS available for Docker.\e[0m"
    echo "At least 2 CPUs recommended. You have ${cpus_available}"
    echo
    warning_resources="true"
fi
if (( disk_available < one_meg * 10 )); then
    echo
    echo -e "\033[1;33mWARNING!!!: Not enough Disk space available for Docker.\e[0m"
    echo "At least 10 GBs recommended. You have $(numfmt --to iec $(disk_available * 1024 ))"
    echo
    warning_resources="true"
fi
if [[ ${warning_resources} == "true" ]]; then
    echo
    echo -e "\033[1;33mWARNING!!!: You have not enough resources to run Airflow (see above)!\e[0m"
    echo "Please follow the instructions to increase amount of resources available:"
    echo "   https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/index.html#before-you-begin"
    echo
fi
echo
echo "Creating missing opt dirs if missing:"
echo
mkdir -v -p /opt/airflow/{logs,dags,plugins,config}
echo
echo "Airflow version:"
/entrypoint airflow version
echo
echo "Files in shared volumes:"
echo
ls -la /opt/airflow/{logs,dags,plugins,config}
echo
echo "Running airflow config list to create default config file if missing."
echo
/entrypoint airflow config list >/dev/null
echo
echo "Files in shared volumes:"
echo
ls -la /opt/airflow/{logs,dags,plugins,config}
echo
echo "Change ownership of files in /opt/airflow to ${AIRFLOW_UID}:0"
echo
chown -R "${AIRFLOW_UID}:0" /opt/airflow/
echo
echo "Change ownership of files in shared volumes to ${AIRFLOW_UID}:0"

```

```

echo
chown -v -R "${AIRFLOW_UID}:0" /opt/airflow/{logs,dags,plugins,config}
echo
echo "Files in shared volumes:"
echo
ls -la /opt/airflow/{logs,dags,plugins,config}

# yamllint enable rule:line-length
environment:
  <<: *airflow-common-env
  _AIRFLOW_DB_MIGRATE: 'true'
  _AIRFLOW_WWW_USER_CREATE: 'true'
  _AIRFLOW_WWW_USER_USERNAME: ${_AIRFLOW_WWW_USER_USERNAME:-airflow}
  _AIRFLOW_WWW_USER_PASSWORD: ${_AIRFLOW_WWW_USER_PASSWORD:-airflow}
  _PIP_ADDITIONAL_REQUIREMENTS: 's3fs boto3'
  user: "0:0"

airflow-cli:
  <<: *airflow-common
  profiles:
    - debug
  environment:
    <<: *airflow-common-env
    CONNECTION_CHECK_MAX_COUNT: "0"
  # Workaround for entrypoint issue. See: https://github.com/apache/airflow/issues/16252
  command:
    - bash
    - -c
    - airflow
  depends_on:
    <<: *airflow-common-depends-on

```

You can enable flower by adding "--profile flower" option e.g. docker-compose --profile flower up
 # or by explicitly targeted on the command line e.g. docker-compose up flower.

See: <https://docs.docker.com/compose/profiles/>

```

flower:
  <<: *airflow-common
  command: celery flower
  profiles:
    - flower
  ports:
    - "5555:5555"
  healthcheck:
    test: ["CMD", "curl", "--fail", "http://localhost:5555/"]
    interval: 30s
    timeout: 10s
    retries: 5
    start_period: 30s
  restart: always
  depends_on:
    <<: *airflow-common-depends-on
  airflow-init:
    condition: service_completed_successfully

```

```

pgadmin:
  image: dpage/pgadmin4
  container_name: pgadmin
  environment:
    PGADMIN_DEFAULT_EMAIL: admin@admin.com
    PGADMIN_DEFAULT_PASSWORD: admin
  ports:
    - "5050:80"
  depends_on:
    - postgres
  restart: always

```

```

volumes:
  postgres-db-volume:

```

Open 3 Git bash Terminals

Terminal 1

```
scp -r -i "YOUR-KEY-PAIR.pem" docker_exp ec2-user@ec2-54-226-198-113.compute-1.amazonaws.com:/home/ec2-user/docker_exp
```

Terminal 2

```
chmod 400 "YOUR-KEY-PAIR.pem"
ssh -i "KEY_PAIR.pem" ec2-user@ec2-54-226-198-113.compute-1.amazonaws.com

ls (docker_exp folder will show)

sudo yum update -y

sudo yum install docker -y

sudo curl -L "https://github.com/docker/compose/releases/download/1.29.1/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose

sudo chmod +x /usr/local/bin/docker-compose

sudo gpasswd -a $USER docker

newgrp docker

sudo yum install python-pip -y

sudo pip install docker-compose

sudo systemctl start docker

docker ps (Check your running containers → 0)

cd docker_exp

docker-compose up -d --build

docker ps (Check your running containers → 2)
```

Terminal 3

```
ssh -i "YOUR-KEY-PAIR.pem" ec2-user@ec2-54-226-198-113.compute-1.amazonaws.com -L 8080:localhost:8080 -L 5050:localhost:5050 -L 5433:localhost:5433 -L 5555:localhost:5555
```

Open these two links

1. Airflow:

<http://localhost:8080/>

- **Username:** airflow
- **Password:** airflow

2. PgAdmin:

<http://localhost:5050/>

- **Username:** admin@admin.com
- **Password:** admin

pgAdmin

Query Tool Workspace

Connect to the Server

Server name: Airflow Postgres
Host name/address: postgres
Port: 5432
Database: airflow
User: airflow
Password: airflow

Connect & Open Query Tool

Create Table in PostgreSQL:

```
CREATE TABLE IF NOT EXISTS weather_data (  
  id SERIAL PRIMARY KEY,  
  city VARCHAR(50),  
  description VARCHAR(100),  
  temperature_f FLOAT,  
  feels_like_f FLOAT,  
  min_temp_f FLOAT,  
  max_temp_f FLOAT,  
  pressure INTEGER,  
  humidity INTEGER,  
  wind_speed FLOAT,  
  time_of_record TIMESTAMP,  
  sunrise TIMESTAMP,  
  sunset TIMESTAMP  
);
```


Airflow connection



Admin → Connection → Add Connection

1. Open Weather

Connection ID : weathermap_api

Connection Type : HTTP

Host: https://api.openweathermap.org

2. Postgres

Connection ID: postgres_default

Connection Type: Postgres

Host: postgres

Login: airflow

Password: airflow

Port: 5432

Database: airflow

Get Access Token:

<code>sudo rm -rf /usr/local/lib/python3.9/site-packages/urllib3*</code>
<code>sudo /usr/bin/python3 -m pip install --upgrade --target /usr/lib/python3.9/site-packages urllib3</code>
<code>sudo yum install awscli</code>
<code>aws configure</code>

- Your User AWS Access Key ID
- Your User AWS Secret Access Key
- Your Default Region name

<code>aws sts get-session-token</code>
--

Copy your Access Key ID, Secret Access Key and Session Token

Write `weather_dag-modify.py` File in DAG folder:

[ec2-user@ip-..... docker_exp] \$ cd dags

[ec2-user@ip-..... dags] \$ sudo nano weather_dag-modify.py

Paste your Access Key ID, Secret Access Key and Session Token in `def transform_load_data()` → `aws_credentials`

Paste your S3 Bucket name in `def transform_load_data ()` → `now`

Paste your Openweather API Key in `is_weather_api_ready` → `endpoint`

Paste your Openweather API Key in `def extract_weather_data ()` → `endpoint`

```

from airflow import DAG
from datetime import timedelta, datetime
from airflow.providers.http.sensors.http import HttpSensor
from airflow.providers.http.hooks.http import HttpHook
from airflow.operators.python import PythonOperator
import json
import pandas as pd
from airflow.providers.postgres.hooks.postgres import PostgresHook

def kelvin_to_fahrenheit(temp_in_kelvin):
    return (temp_in_kelvin - 273.15) * (9 / 5) + 32

# ----- Replace SimpleHttpOperator with this -----
def extract_weather_data(**context):
    http_hook = HttpHook(method='GET', http_conn_id='weathermap_api')
    response = http_hook.run(
        endpoint='/data/2.5/weather?q=Portland&APPID='
    )
    data = response.json()
    context['ti'].xcom_push(key='weather_data', value=data)

def transform_load_data(task_instance):
    data = task_instance.xcom_pull(key='weather_data', task_ids='extract_weather_data')

    city = data["name"]
    weather_description = data["weather"][0]["description"]
    temp_fahrenheit = kelvin_to_fahrenheit(data["main"]["temp"])
    feels_like_fahrenheit = kelvin_to_fahrenheit(data["main"]["feels_like"])
    min_temp_fahrenheit = kelvin_to_fahrenheit(data["main"]["temp_min"])
    max_temp_fahrenheit = kelvin_to_fahrenheit(data["main"]["temp_max"])
    pressure = data["main"]["pressure"]
    humidity = data["main"]["humidity"]
    wind_speed = data["wind"]["speed"]
    time_of_record = datetime.utcfromtimestamp(data['dt'] + data['timezone'])
    sunrise_time = datetime.utcfromtimestamp(data['sys']['sunrise'] + data['timezone'])
    sunset_time = datetime.utcfromtimestamp(data['sys']['sunset'] + data['timezone'])
    # ----- Save to S3 -----
    transformed_data = {
        "City": city,
        "Description": weather_description,
        "Temperature (F)": temp_fahrenheit,
        "Feels Like (F)": feels_like_fahrenheit,
        "Minimum Temp (F)": min_temp_fahrenheit,
        "Maximum Temp (F)": max_temp_fahrenheit,
        "Pressure": pressure,
        "Humidity": humidity,
        "Wind Speed": wind_speed,
        "Time of Record": time_of_record,
        "Sunrise (Local Time)": sunrise_time,
        "Sunset (Local Time)": sunset_time
    }

    df_data = pd.DataFrame([transformed_data])
    aws_credentials = {
        "key": "",
        "secret": "",
        "token": ""
    }

```

```

now = datetime.now()
dt_string = now.strftime("%d%m%Y%H%M%S")
file_name = f"current_weather_data_portland_{dt_string}.csv"
df_data.to_csv(f"s3://s3-openweather-api/{file_name}", index=False, storage_options=aws_credentials)

# ----- Save to PostgreSQL -----
postgres_hook = PostgresHook(postgres_conn_id='postgres_default')
insert_query = """
    INSERT INTO weather_data (
        city, description, temperature_f, feels_like_f,
        min_temp_f, max_temp_f, pressure, humidity,
        wind_speed, time_of_record, sunrise, sunset
    ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
    """

record_to_insert = (
    city, weather_description, temp_fahrenheit, feels_like_fahrenheit,
    min_temp_fahrenheit, max_temp_fahrenheit, pressure, humidity,
    wind_speed, time_of_record, sunrise_time, sunset_time
)
postgres_hook.run(insert_query, parameters=record_to_insert)

# ----- DAG Definition -----
default_args = {
    'owner': 'airflow',
    'depends_on_past': False,
    'start_date': datetime(2023, 1, 8),
    'email': ['airflow@gmail.com'],
    'email_on_failure': False,
    'email_on_retry': False,
    'retries': 2,
    'retry_delay': timedelta(minutes=2)
}

with DAG(
    'weather_dag',
    default_args=default_args,
    schedule='@daily',
    catchup=False
) as dag:

    is_weather_api_ready = HttpSensor(
        task_id='is_weather_api_ready',
        http_conn_id='weathermap_api',
        endpoint='/data/2.5/weather?q=Portland&APPID='
    )

    extract_weather_data = PythonOperator(
        task_id='extract_weather_data',
        python_callable=extract_weather_data
    )

    transform_load_weather_data = PythonOperator(
        task_id='transform_load_weather_data',
        python_callable=transform_load_data
    )

    is_weather_api_ready >> extract_weather_data >> transform_load_weather_data

```

Check Airflow Dags:

weather_dag → Success (Dark Green)

The screenshot shows the Airflow web interface for a DAG run. The DAG is named 'weather_dag' and was triggered on 2025-10-27 01:27:41. The run status is 'Success'. The task list shows three tasks: 'is_weather_api_ready', 'extract_weather_data', and 'transform_load_weather_data', all of which are in a 'Success' state.

Task ID	Map Index	State	Start Date	End Date
is_weather_api_ready		Success	2025-10-27 01:27:41	2025-10-27 01:27:45
extract_weather_data		Success	2025-10-27	2025-10-27

Check pgAdmin:

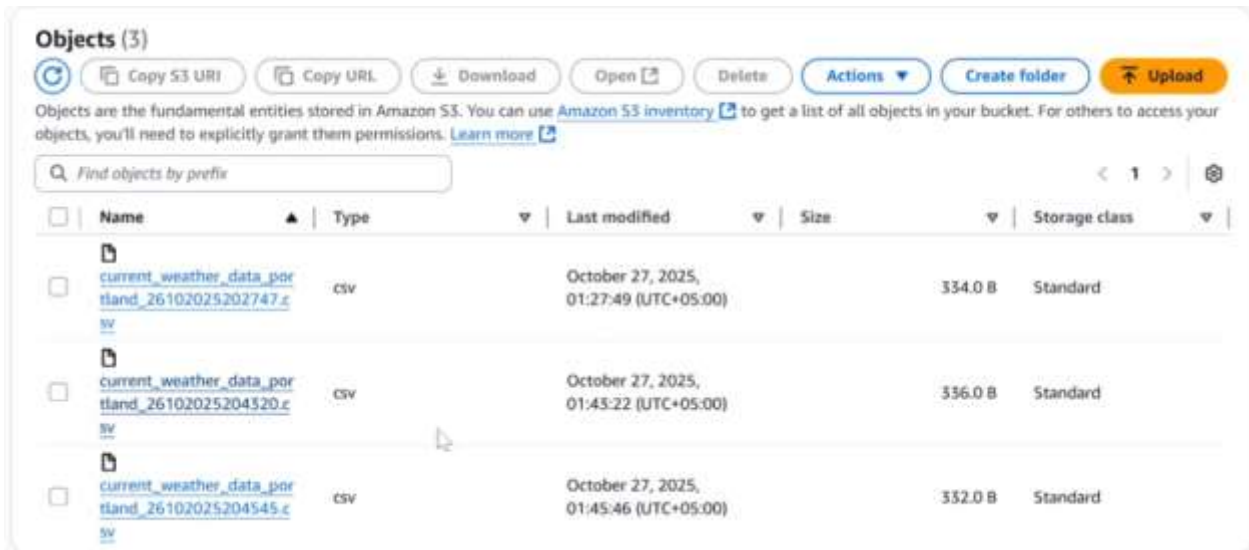
SELECT * FROM weather_data:

The screenshot shows the pgAdmin interface with the query 'SELECT * FROM weather_data' executed. The results are displayed in a table with 4 rows and 11 columns. The status bar at the bottom indicates 'Successfully run. Total query runtime: 613 msec. 4 rows affected.'

id	city	description	temperature_f	feels_like_f	max_temp_f	max_humid_f	pressure	humidity	wind_speed
1	Portland	light rain	48.48800000000004	44.438000000000043	46.778000000000096	49.948000000000055	1012	93	4.1
2	Portland	light rain	48.48800000000004	44.438000000000043	46.778000000000096	49.948000000000055	1012	93	4.1
3	Portland	moderate rain	48.48800000000004	44.438000000000043	46.778000000000096	49.948000000000055	1012	93	4.1
4	Portland	light rain	48.48800000000004	44.438000000000043	46.778000000000096	49.948000000000055	1012	93	4.1

Check S3 Bucker:

s3-openweather-oh → csv files



The screenshot displays the Amazon S3 console interface for a bucket named 's3-openweather-oh'. At the top, there are buttons for 'Copy S3 URI', 'Copy URL', 'Download', 'Open', 'Delete', 'Actions', 'Create folder', and 'Upload'. Below these buttons, a text box explains that objects are fundamental entities stored in Amazon S3 and provides a link to 'Amazon S3 inventory' for a list of all objects. A search bar labeled 'Find objects by prefix' is also present. The main area shows a table of three objects, all of which are CSV files. Each object's name follows the pattern 'current_weather_data_portland_26102025202747.csv', 'current_weather_data_portland_26102025204320.csv', and 'current_weather_data_portland_26102025204545.csv'. The table also lists the 'Last modified' date and time, the 'Size' in bytes, and the 'Storage class' as 'Standard'.

☐	Name	Type	Last modified	Size	Storage class
☐	current_weather_data_portland_26102025202747.csv	csv	October 27, 2025, 01:27:49 (UTC+05:00)	334.0 B	Standard
☐	current_weather_data_portland_26102025204320.csv	csv	October 27, 2025, 01:43:22 (UTC+05:00)	336.0 B	Standard
☐	current_weather_data_portland_26102025204545.csv	csv	October 27, 2025, 01:45:46 (UTC+05:00)	332.0 B	Standard